Project in Pervasive Computing - Sheev

Stefan Haslhofer

Contents

1 Overview

Sheev is a small autonomous vehicle with four wheels that tracks people in its vicinity once activated. It uses a machine learning library provided by Google on a smartphone to recognize humans via a camera. The vehicle starts to operate after an activation action, in my case, opening a door. A picture of the vehicle can be seen in Figure 1.

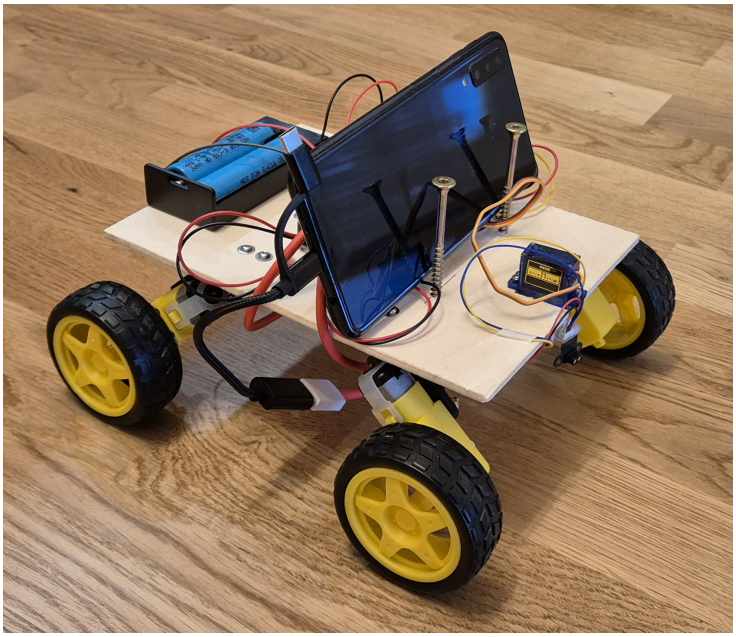


Figure 1: Picture of the vehicle.

2 Architecture

Sheev consists of a vehicle and an external door sensor as described in Figure 2. The vehicle itself is then subdivided into the vehicle body and the smartphone which sits on top.

The **vehicle body** (1) consists of four wheels and a basic wooden plate that holds all the electronic parts needed for steering and obstacle detection. We will have a closer look at the parts used in subsection 2.1. The **smartphone** (2) on top of the vehicle body is used for human detection and communication with the door sensor. It tells the vehicle body when a door has been opened and where a detected person is located. The vehicle body then adjusts its behavior according to the information received from the smartphone. The **door sensor** (3) is mounted on a door and sends out a broadcast through the local network if an opening action is detected.

2.1 Vehicle

The following list contains all major parts used to build the vehicle body:

Part	Usage
ATmega328P Microcontroller	steers car according to input data
L293D motor driver shield	serves as H-bridge, simplifying bidirectional control of DC motors
2x 10A li-ion-battery, 3,6V	power supply
4x DC TT-motor and wheels	driving
TOF laser range sensor	obstacle detection
SG90 servo motor	perform frontal sweeps with laser range sensor for larger field of view

Table 1: Parts used in the vehicle body.

As mentioned above, the vehicle is a basic wooden plate on four wheels. All electronics are screwed on top of the plate. The laser range finder mounted on the servo motor is located in the front. Each motor is mounted on a custom

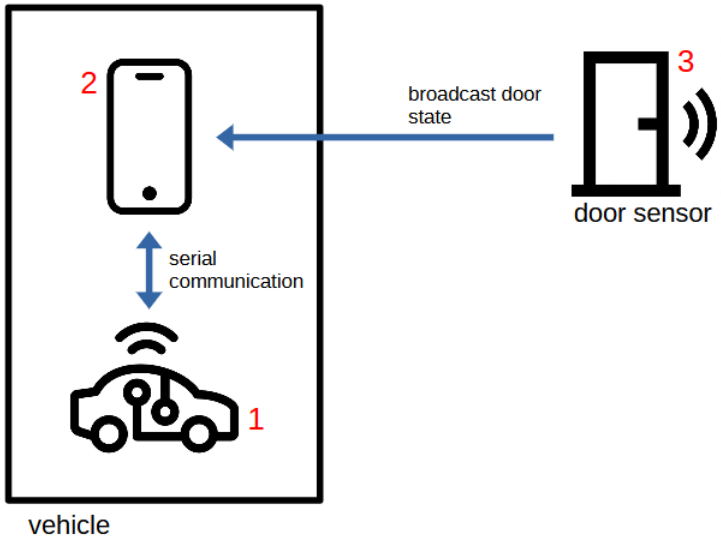


Figure 2: Overall system architecture including basic interactions of all parts. (icons: www.flaticon.com)

3D-printed bracket, angled to optimize ground clearance, and therefore reducing interference with terrain. The batteries are positioned on the rear side.

2.2 Smartphone

The smartphone acts as a middle man for communication between the peripheral door sensor and the micro-controller steering the vehicle. I implemented an *Android* app using *Kotlin* listening to network broadcasts sent by the door sensor. The technical details of the door sensor are described in subsection 2.3. When a door-open event is received the app uses the smartphone's camera to analyze the live feed and subsequently detect humans using a pretrained AI model from *Google's ML Kit*.

Upon successfully detecting a human being, the smartphone sends the position of the person via a serial interface to the micro-controller. The micro-controller processes this data to adjust the driving direction such that the vehicle follows the person. The data flow between the smartphone and the micro-controller is examined in subsection 3.2.

I used the smartphone to reduce costs, eliminating the need to buy a camera. Moreover, the micro-controller used for driving the vehicle has not enough computational power to support image processing with machine learning models at a high enough rate. The smartphone also made setup easier, because it conveniently has its own power supply, a serial port, and is equipped with Wi-Fi. Additionally, I wanted to create a mobile app and try out *Kotlin* but had neither found the time nor a suitable use case until now.

2.3 Door Sensor

The door sensor consists of a simple reed-switch connected to an *ESP8266* micro-controller. I used a *ESP8266* variant with a built-in Wi-Fi module. To activate the reed switch, a magnet was glued to the door. In a closed state the magnet aligns with the reed switch exposing it to a magnetic field that draws the two contacts together, closing the circuit. When the door gets opened the magnet is moved away from the reed switch causing the contacts to separate again and successively open the circuit. The micro-controller reacts to that state change and broadcasts the encoded door status to the local network.

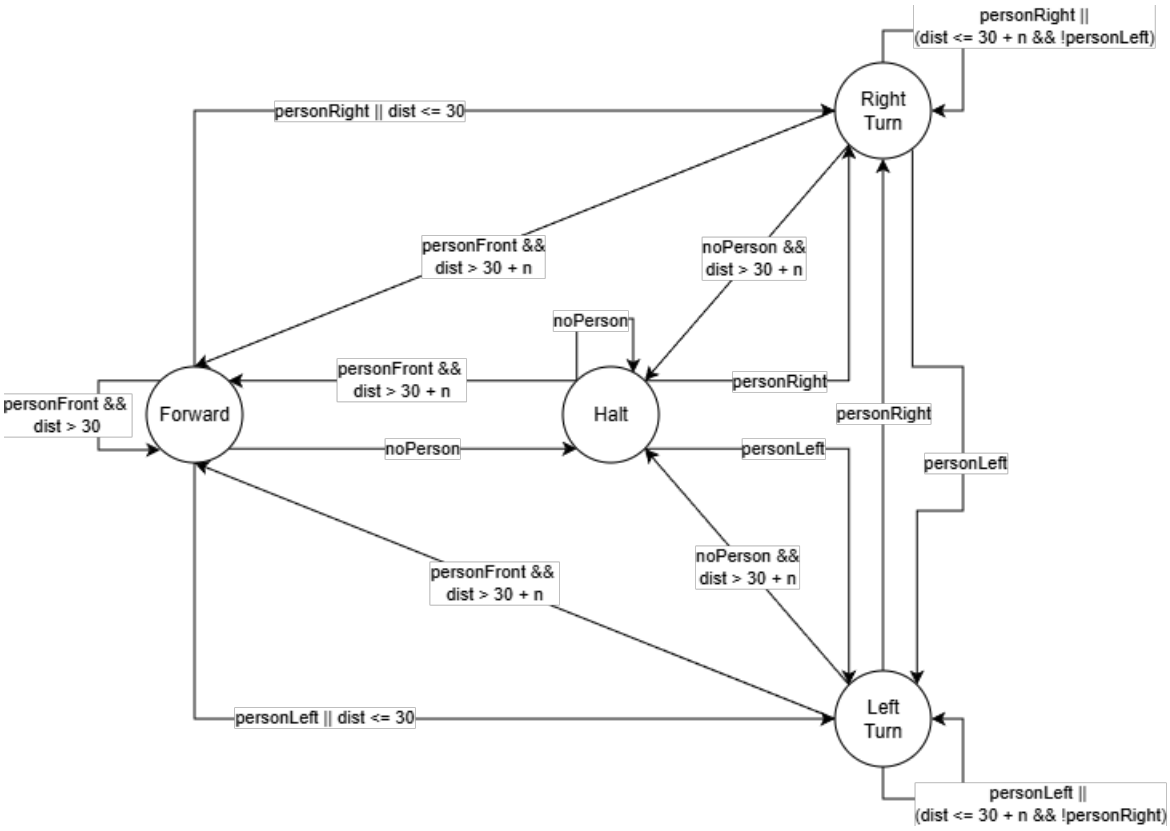


Figure 3: Vehicle behavior outlined as state graph.

3 Implementation

This section covers the implementation of core features in detail explaining the logic and highlighting design decisions.

3.1 Vehicle Behavior

The vehicle operates using four movement states: *forward*, *right turn*, *left turn*, and *halt*. Upon detecting an obstacle or a person, the vehicle dynamically transitions between these states according to the state machine illustrated in Figure 3. Examining the graph, it becomes evident that the vehicle's behavior is not fully deterministic. If an obstacle is detected during forward movement, the turn direction is selected randomly. Obstacle detection is explained in more detail in subsection 3.3. The vehicle operates at a fixed predefined speed.

3.2 Human Detection

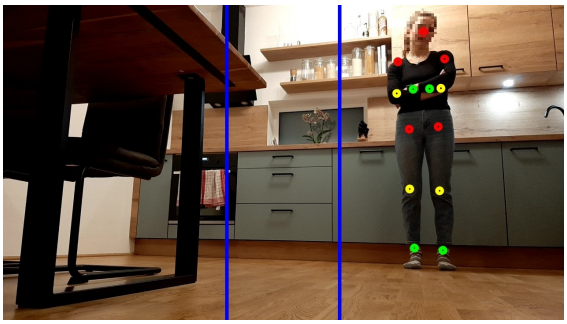
I implemented an Android app written in *Kotlin* that uses *Google's ML Kit* for human detection. At first, I tried to apply a *TensorFlow* model, which proved to be much more complicated than expected. Therefore, I switched to *ML Kit's* pose detection, which is a lightweight solution that works with Android out of the box.

Initially, the person detection model's performance was insufficient for real-time applications. To address this, I reduced the video resolution, which significantly decreased processing lag. Each video frame is now forwarded to *ML Kit's* pose detector, which extracts body features (i.e. *landmarks*) along with their pixel coordinates in the image. Using the detected body feature positions, the smartphone transmits driving instructions over the serial interface to the micro-controller as a status code:

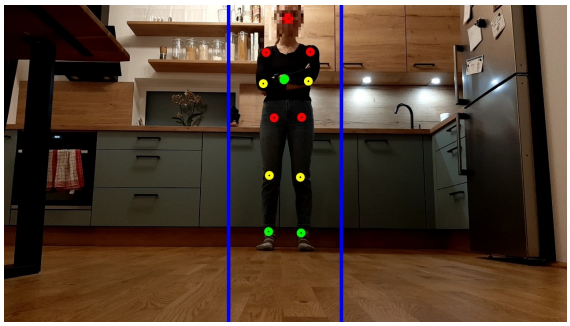
- **0** if all landmarks of the detected body are to the right of a predefined right boundary (Figure 4a).
- **1** if the detected body is fully or partially within the predefined boundaries (Figure 4b).
- **2** if all landmarks of the detected body are to the left of a predefined left boundary (Figure 4c).

- **3** if no person has been detected.

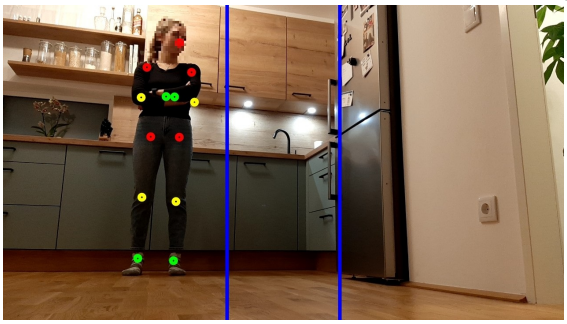
Serial communication is implemented using the *UsbSerial* library, available at `https://github.com/felHR85/UsbSerial`. The library simplifies implementing the USB-to-serial functionality by providing built-in support for *CH340* chips, which are used on the micro-controller.



(a) Detected person right of boundaries.



(b) Detected person within boundaries.



(c) Detected person left of boundaries.

Figure 4: First person view of the vehicle, with critical movement boundaries highlighted as blue lines. Dots mark detected body landmarks.

3.3 Collision Detection

Obstacle detection is performed using a front-mounted laser range finder, which continuously measures and reports the distance to the nearest object. If an object is within a *stop* distance d_s the vehicle immediately halts for 250ms and starts turning in a random direction until no object is detected within *go* distance d_g, where $d_g = d_s + n$ and n is a safety threshold. Without this threshold, the vehicle gets stuck in a cycle of small forward motions and immediate halts when facing a wider obstacle such as a wall, slowing down evasive maneuvers drastically.

Initially, the sensor was mounted in a static forward facing position. Obviously, this limited the ability to react to obstacles not lying directly on the current path. Because of that, the vehicle often got stuck in corners and armchair legs during testing. To expand the object detection coverage area, I attached the sensor to a servo motor that performs horizontal sweeps across the vehicle's front field of view. This significantly increased the vehicle's ability to avoid collisions.

I2C is used for communication between the sensor and the micro-controller. The sensor transmits data in 256-byte packets, which get stored in a buffer. Due to hardware limitations I encountered, which were thankfully stated in the manufacturers data sheet, the data had to be read in two separate 128-byte chunks. Once the buffer is full we are able to extract the distance measurement as shown in Listing 1. According to the manufacturer's specification the value determining the distance is a 32-bit integer spanning bytes *0x24* to *0x27*, with *0x27* being the most significant byte (little endian format). The value must be reconstructed by concatenating these bytes in the correct order.

```
uint32_t  dist  = (unsigned long)(
    ((unsigned long)read_buf[TOF_ADDR_DIS + 3] << 24) |
    ((unsigned long)read_buf[TOF_ADDR_DIS + 2] << 16) |
    ((unsigned long)read_buf[TOF_ADDR_DIS + 1] << 8) |
    (unsigned long)read_buf[TOF_ADDR_DIS]
);
```

Listing 1: Extracting the distance from the buffer.

4 Testing and Possible Improvements

There are some known issues and suboptimal design choices that need to be addressed in order to achieve a performance suitable for real world application. For example, the stationary smartphone camera limits the vehicle's ability to track a moving person. A better approach would be to use a dedicated camera mounted on two servo motors. This would allow the camera to follow a person along x-axis and y-axis. The driving direction could then be adjusted according to the vector alignment of the camera.

This solution would also address a second issue: when a person moves too close to the camera, they may fall outside the field of view. By enabling dynamic camera movement, a tilt mechanism would ensure a target remains visible. However, this would have required upgrading hardware, which was ruled out due to cost and availability. Also, I would not have been able to try out Kotlin.

I also conducted tests with two persons in the field of view. The vehicle chose to follow the most prominent and did not alternate heavily between two directions. Lighting proved to be a more severe problem as it led to frequent missed detections, while it unexpectedly also introduced many false positives.

Although there is significant room for improvement, the project ultimately achieved the main goals and (which is far more important) was fun to implement.